

Concurrency Safety in C++ Data Structures

Committee: ISO/IEC JTC1 SC22 WG21 SG1 Concurrency

Document Number: P0495r0

Date: 2016-11-27

Authors: Lawrence Crowl

Reply To: Lawrence Crowl, Lawrence@Crowl.org

Problem

Synchronization between operations is an important property of concurrent data structures. The correct use of these data structures requires a specification of their synchronization.

The simplest concurrent data structures in C++ are the atomic types. The atomic types take a function parameter to specify memory order. They provide query functions returning whether they are lock-free or not.

Larger data structures are unlikely to be able to efficiently take a function parameter. Indeed, the data structure is often very carefully designed to satisfy a given level of synchronization.

More importantly, atomic types are *implementation types*, generally used as a direct tool the implementation of other abstractions. For larger data structures, we often want a concept that enables different implementations.

How do we specify synchronization in complex C++ data structures so as to ensure that they are used safely?

Memory Order

The atomic types take a function parameter to specify memory order. Their representation does not change depending on the memory order, so this specification works well for atomic types. For more complex data structures, the representation may change.

Since most concurrent data structures will be implemented as class templates, the strawman approach is to add a type parameter to the class template to specify the memory order. The library would then use the template parameter as a specialization key for selecting the data structure implementation.

Unfortunately, that type parameter is not visible when the data structure itself becomes a parameter to another template. Generally, this loss of information is a good thing. It abstracts the essence of the service that the data structure provides, allowing the concrete data structure to be easily replaced as needs evolve.

Unfortunately, the memory order guaranteed by a concurrent data structure generally must be visible. Or, more to the point, any order less than sequentially consistent requires care and should be obvious to the programmer. This implies that template parameter to the data structure is not an adequate assurance of proper use of non-sequentially consistent data structures.

The only mechanism to 'type check' a template type parameter is the names and parameters of the operations applied to that parameter. So, to catch an unexpected use of non-sequentially-consistent data structures as a template parameter, we must make the function signatures different from a sequentially-consistent signatures.

The obvious signature change is to change the name of the functions. We can do better. Make the operations function templates with the required memory order as a template parameter. A default of sequentially consistent will serve most programmers, who require the simplicity of sequential consistency, without imposing any additional burden on the text of the program.

Clients requiring less than sequential consistency can be satisfied with an implementation providing sequential consistency. So, those implementations can essentially ignore the template parameter in their implementations of operations.

What memory orders make sense in larger data structures?

Most operations both read and write. However, we do not want to call them read-modify-write operations unless they meet the constraints of such operations.

Other Issues

Do we need to specify lock freedom? Wait freedom? Any thing else?